

Module 5: Software Assurance and Secure SQL

Denver Clarke (dclar106), EN.605.256. Verified September 30, 2026.

Installation and packaging

Use Python 3.12, the verified tooling/CI version. From module_5/, create a virtual environment with `python -m venv .venv`, then run `.venv/Scripts/python -m pip install -r requirements.txt` and `.venv/Scripts/python -m pip install -e .` on Windows. As an alternative, run `uv venv .venv`, `uv pip sync --python .venv/Scripts/python.exe requirements.txt`, and `uv pip install --python .venv/Scripts/python.exe -e .`. The `setup.py` metadata makes the local modules installable in editable mode, so imports work consistently in development, tests, and CI. Both install methods have been verified locally with successful `app`, `models`, and `query_data` imports. After configuring the database environment variables, run `.venv/Scripts/python src/run_flask.py` and open `http://127.0.0.1:5000/analysis`. On Linux/macOS, use `.venv/bin/python`.

SQL injection defenses and query limits

The analysis reads in `query_data.py` now have explicit `LIMIT` clauses. The SQLAlchemy analysis statements also use `.limit()` and bind their values through SQLAlchemy expressions. The new `lookup_applicants` function accepts only an approved search column, quotes the table and column with `psycopg.sql.Identifier`, and passes the search value through a placeholder and a separate parameter tuple. The SQL text never contains the user's value. The requested row count is clamped to the range 1 through 100, and the limit is passed as a bound parameter. A test with `Accepted ' OR 1=1 --` verifies that the malicious value stays in the parameter tuple and never enters the SQL statement. The loader also uses `sql.Identifier` when it needs to compose a database name. The existing Flask analysis routes do not accept SQL search filters; the lookup helper is a separately tested safe-search API, not a new web endpoint. The analysis endpoint safely ignores malicious query values.

Least privilege database role

The application now reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD` from the environment, with `DATABASE_URL` and legacy `PG*` variables as alternatives. The example file `.env.example` has placeholders, and `.gitignore` excludes `.env`. The `gradcafe_app` role receives only `CONNECT` on the database, `USAGE` on schema `public`, `SELECT` and `INSERT` on `public.applicants`, and `USAGE` on the primary key sequence. `SELECT` is needed for analysis; `INSERT` and sequence use support the Pull Data action. It does not own the database or receive `CREATE`, `ALTER`, `DROP`, `UPDATE`, `DELETE`, or superuser privileges. The local grant script is `least_privilege.sql`. Read-only connections no longer try to provision a database. The data loader must be run separately with an administrator role when it creates the schema.

This role was created and tested in the disposable PostgreSQL 17 CI database `gradcafe_test`, using `tools/verify_least_privilege.py`. The captured output in `database_privileges.txt` confirms denied DDL/destructive writes, successful `SELECT/INSERT`, safe malicious-input handling, and working Flask analysis and update responses. No home database changes were made: local deployment still requires an administrator to run `setup_database.cmd` and set private app credentials. CI generates a random, temporary role password without printing it.

The CI grant statements include `GRANT CONNECT ON DATABASE gradcafe_test TO gradcafe_app;`, `GRANT USAGE ON SCHEMA public TO gradcafe_app;`, `GRANT SELECT, INSERT ON TABLE public.applicants TO gradcafe_app;`, and `GRANT USAGE ON SEQUENCE public.applicants_p_id_seq TO gradcafe_app;`.

Dependency graph (seven sentences)

`dependency.svg` was generated from `src/app.py` with `pydeps` and `Graphviz`. The `app` module defines Flask routes and asks `orm_queries` for the analysis shown on the page. `orm_queries` uses `models` for the applicant mapping and SQLAlchemy session access. `models` depends on SQLAlchemy and the PostgreSQL dialect, which connect the Python model to PostgreSQL. `scrape_refresh` joins the scraper, data loader, and model layer for the optional refresh action. `analysis_format` turns database results into display strings. Flask and its template helpers render the page, while the external SQLAlchemy and Flask nodes account for most of the graph's complexity.

Verification and CI

The local Module 5 test command passed with 132 tests, three skips, and 100% coverage. Pylint on `src/` returned 10.00/10 using `python -m pylint src --fail-under=10`. The repository root workflow `.github/workflows/ci.yml` has separate Pylint, dependency graph, Snyk, and Pytest jobs. It runs on every push and pull request, with a manual trigger available. The Snyk job requires the `SNYK_TOKEN` repository secret. All four jobs passed in GitHub Actions run 36719680940 (commit 87284d8); `ci_success.jpg` captures the successful run. Hosted Pytest passed 133 tests, with two optional model tests skipped and 100% coverage, including the live PostgreSQL test. The hosted Snyk scan tested 50 Linux dependencies with zero issues. The existing Module 4 Read the Docs site remains online; Module 5 Sphinx HTML also builds locally with warnings treated as errors.

Snyk status

Snyk CLI 1.1307.4 successfully scanned `requirements.txt` after browser authentication and installation of the project dependencies. The scan completed with exit code 0 and tested 52 dependencies; it reported zero issues and no vulnerable paths. The original output is saved in `snyk-analysis.txt`, and the machine-readable result is `snyk-results.json`. The screenshot `snyk-analysis.png` shows the saved CLI output in a browser. This result describes known dependency issues at scan time, not a guarantee that the application has no security defects. No dependency remediation was required by this scan.

Extra credit: Snyk Code (SAST)

After enabling Snyk Code with the account owner's approval, I ran `snyk code test --json-file-output=snyk-code-results.json` from `module_5/`. The initial scan found 26 issues: zero high, three medium, and 23 low. I fixed the medium XSS finding in the optional capture receiver by escaping the filename in its HTML success message, and also escaped the form token. A focused output-escaping check passed. A repeat scan confirmed the XSS finding was removed: 25 findings remain (zero high, two medium, 23 low).

The two remaining medium findings concern operator-selected local Python and browser executables. Both use argument lists without a shell, but the executable and package inputs still require a trusted local operator. The low findings comprise 22 local file-path warnings and one fixed CI username warning; the CI password is randomly generated, not hardcoded. These findings remain open and unsuppressed. They are not evidence of a clean source scan or a proven remote exploit. Use trusted CLI inputs and do not expose these helpers as multi-user services without additional containment.

Both scans completed with findings (exit code 1). Genuine before/after CLI output is in `snyk-code-initial.txt` and `snyk-code-analysis.txt`; both SARIF JSON files are included. `SNYK_CODE_REVIEW.md` explains the remediation and remaining trust boundaries. This completes the requested SAST execution, output evidence, and findings summary for extra-credit review.